

Emilio Sanchez San Martin
Information Science & +1 Smith Business Analytics Student
University of Maryland, College Park

Published June 10, 2026

***Author Note:** This instructional guide was developed at the University of Maryland, College Park, and integrates principles of database design and technical communication to help beginner students understand the process of normalizing datasets through Third Normal Form (3NF). The guide draws upon concepts learned in INST327 (Database Design and Modeling) and ENGL393 (Technical Communication).*

Audience Analysis

Many students at the University of Maryland are currently studying computer science or information science-related concepts, and many have an interest in working with data. My instructions will benefit those who want to understand how to work with databases, specifically for those taking classes in data design, and get through the first concept of data designing, which is normalization. I myself am part of this group of students who have worked with large datasets and struggled to understand how to properly structure them for a database. While I have much experience working in SQL and working with data, I don't hold a formal role, but I have gained enough knowledge to help others understand the foundational concept of normalization.

I am writing these instructions for students who are taking data design or SQL classes and are new to databases, and may need help understanding how to normalize data once they know the different forms and relationships in databases. These students will often appreciate clear step-by-step explanations and visual examples since technical topics can be overwhelming and hard to follow without proper guidance. Many may also be motivated to build their skills to land internships or future career opportunities.

These students, in my experience, may struggle as I did in understanding normalization, as it can often be shown in theoretical ways without showing how to apply them in real datasets. My goal is to provide a clear, practical process to transform the data into a structured format to help better understand how databases are designed and used when first normalizing data.

Normalizing a Dataset to Third Normal Form (3NF) Using Excel: Step-by-Step Guide for Beginner UMD Data Science Students

Background

Information scientists often learn about datasets in their second year of studies at UMD. Many students, however, struggle with doing one of the most important early steps in database design: normalizing. While there are students who know how to work with data and understand how to analyze it, many don't know how to properly structure different tables in data to make organized tables that can be used in a relational database. These instructions are helpful for students who want to learn more about making new databases and understand how to transform unstructured data into a structured format with the correct tables. In this guide, you will learn how to normalize a dataset step-by-step up to the Third Normal Form (3NF) utilizing Microsoft Excel. At the end, you will understand the process of breaking down complex datasets and making relationships between various tables and be prepared to use data for database systems in software such as MySQL.

Technical Background

Database normalization is a process in order to organize data in different structured tables and improve databases. It is very important, as it helps the database design process and improves the database's efficiency, consistency, and accuracy. Many of those working in databases and programs such as SQL use this process, making it easier to manage and maintain data so that a database is adaptable and can make changes for business, governmental, or healthcare needs. We organize 'attributes' of the database to get rid of data redundancy (same data in different places).

Let's say if we were to keep this, it can unnecessarily increase the size of the database, as the same data is repeated in various places. You wouldn't be able to run queries for a database with inconsistency, errors to find data would occur, and data would be messed up!

Normalization of a database is achieved by following a set of rules called 'forms' in creating the database. When breaking down large, unstructured datasets into smaller, related tables, we use common levels of normalization such as First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). With each step, it improves how data are stored and related to one another.

Normalization generally involves having to split a single table of data into multiple different ones, which are all linked each time a query is made, which takes data from the split tables. Once

they are split, you can start adding those tables to Excel and have separate tables/data to apply in your database.

Another concept that's important in normalization is the use of primary keys (PK) and foreign keys (FK), which you will see in this process. A primary key is a unique identifier for each record in a table, while a foreign key is used to link one table to another. These keys allow different tables to relate to each other without duplicating data.

These instructions will assume that the user has a basic understanding of using Microsoft Excel, which includes making tables and organizing data. No prior experience in SQL or advanced database design is needed, as these instructions focus specifically on the normalization process using real data as an example. Dependencies and normalization forms are the main details that will be helpful to learn about prior to reading these instructions.

Warnings

- **Do not skip steps in the normalization process.** Each normal form gets built off the previous form, and skipping steps can result in incorrect table structures.
- **Watch out for misidentifying dependencies.** Misidentifying fields that depend on others can make wrong table separations.
- **Avoid deleting data.** Make sure to keep a copy of the original dataset before making changes, in case mistakes happen.
- **Make sure column names are consistent when splitting tables** to avoid confusion.
- **Do not assume relationships without verifying.** Always check to see how data is related before creating primary or foreign keys in different tables.

Materials

1. Computer or Laptop

A computer capable of running Microsoft Excel is required to follow these instructions and organize data into structured tables.

2. Microsoft Excel (Microsoft 365 or Excel 2019+)

Excel will be used to view, organize, and separate the dataset into multiple tables. Users should be familiar with basic Excel functions such as creating sheets, sorting data, and editing columns.

3. Sample Dataset File With Data/Correct Columns (Provided in Excel Format)

A simplified version of the Montgomery County Police Department crime dataset will be used for this guide. This dataset includes selected columns such as...

{Incident ID, Offense Code, Victims, Crime Against, Crime Name, Crime Category, Police District Name, Police District Number, City, State, Zip Code, and Agency}.

- *Example of a CSV Dataset:*

Image 1.

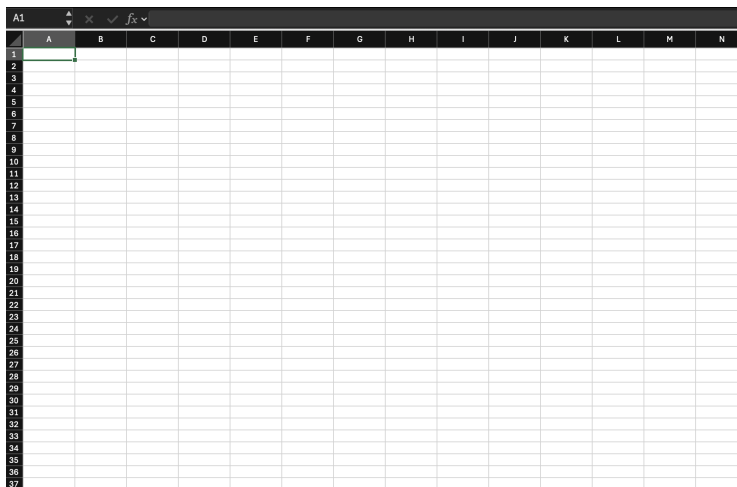
Instructions

Section 1 – Setting Up Your Dataset in Excel

1: Open a New Excel Workbook

- Open Microsoft Excel on your computer
- Click "File," then click "New" to create a new file

Result: You will have an empty spreadsheet ready for your dataset.



2: Input Column Headers (Unnormalized Data)

- REFERENCE BACK to your original dataset [image 1]. For example, with my dataset I have...

{Incident_ID | Offense Code | Victims | Crime Against | Crime_Name | Crime_Category | Police District Name | Police District Number | City | State | Zip Code | Agency}

- In **Row 1**, enter the following column names above and highlight **Incident_ID** in **Red**.

- Why?:* The **red highlight** identifies the **Primary Key (PK)** of your original dataset

- ii. Since this ID is the unique identifier of a specific 'crime report' (in your case, you would do this for the data you are working with), highlighting it now helps you track how this specific piece of data **moves** and **connects to other tables** as we normalize the database. We will highlight in colors PK and FK as we normalize, serving this as our 'Visual Key.'
- iii. These columns represent your **original unstructured dataset**.

3: Add a Sample Data Row

- a. In **Row 2**, input one example row from your dataset.
Tip: Only use **one row** for learning purposes. (*This is to help you visually understand relationships and prevent confusion for large datasets.*)

4: Create a Vertical Attribute List

- a. In **Column A (starting below your table)**, list each attribute:

	A	B
1	Incident_ID	Offence Code V
2	201087100	1315
3		
4		
5	Data:	
6	Incident ID	
7	Offence Code	
8	Victims	
9	Crime_Against	
10	Crime_Name	
11	Crime_Category	
12	Police District Name	
13	Police District Number	
14	City	
15	State	
16	Zip Code	
17	Agency	
18		

This helps you...

- Identify Dependencies
- Plan how to split tables

Results thus far:

HomeInsertDrawPage LayoutFormulasDataReviewViewAutomate

Paste

Aptos Narrow (Bod...11

B

I

U

General

\$

%

0.00

→0

Conditional Formatting

Format as Table

Cell Styles

Insert

Delete

Forma

M10

	A	B	C	D	E	F	G	H	I	J	K	L
1	Incident_ID	Offence Code	Victims	Crime Against	Crime_Name	Crime_Category	Police District Name	Police District Number	City	State	Zip Code	Agency
2	201087100	1315	1	Crime Against Per	Aggravated Assa	Aggravated	GERMANTOWN	5D	GERMANTO	MD	20876	MCPD
3												
4												
5	Data:											
6	Incident ID											
7	Offence Code											
8	Victims											
9	Crime_Against											
10	Crime_Name											
11	Crime_Category											
12	Police District Name											
13	Police District Number											
14	City											
15	State											
16	Zip Code											
17	Agency											
18												

Section 2 – First Normal Form (1NF)

5: Identify Logical Groupings of Data

- Review your columns and group related attributes:
 - Incident-related data
 - Offense-related data

Background (What First Normal Form Does):

1NF is the process of separating data into tables. Each column contains only one value, and each record can be uniquely identified. The goal of 1NF is to “remove” grouped information and begin organizing the dataset into a “structure” that can later be improved (GeeksforGeeks).

In this step, the dataset begins moving from *one large table* into *smaller tables* that represent separate types of information.

6: Create the “Incidents” Table

- Create a new table labeled

[Incidents]

- Incident_id (PK)
- Victims
- Police District Number
- Police District Name
- City

- State
- Zip Code
- Agency

Action in Excel:

- Copy relevant columns into a new section
- Highlight header row → apply **highlight in yellow**
- Incident_id (PK) **red**

7: Create the “Incident Offense” Table

- Another table labeled

[Incident_Offense]

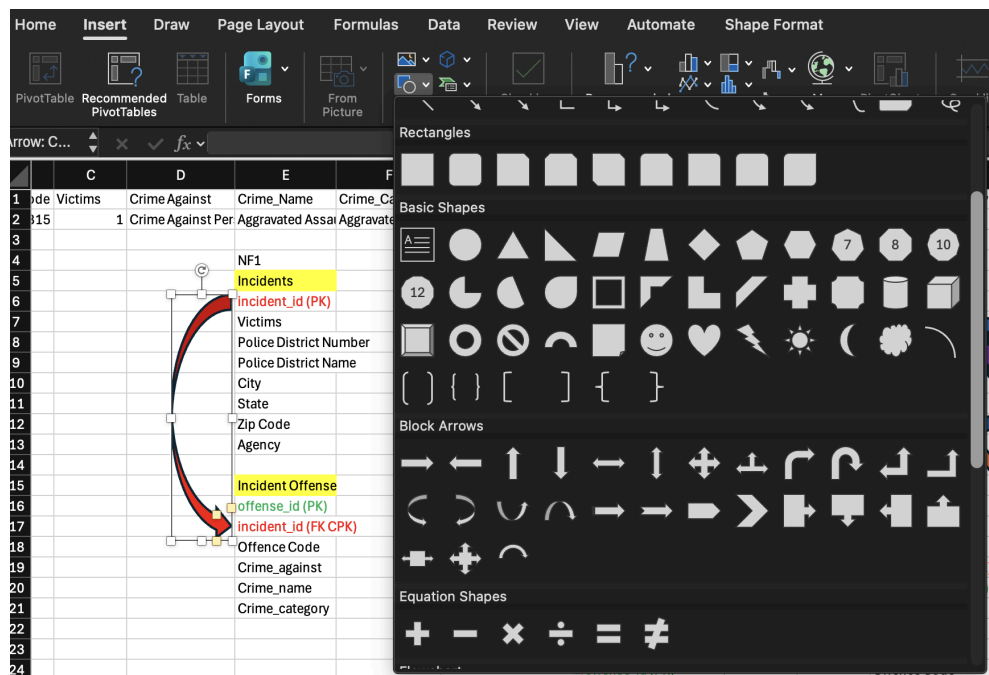
- Offense_id (PK)
- Incident_id (FK)
- Offense Code
- Crime_against
- Crime_name
- Crime_category

8: Define Primary and Foreign Keys

- If not already done so, as shown above, make sure you define the primary and foreign keys with the color code **Visual key**.
 - incident_id as Primary Key (PK)** in Incidents
 - offense_id as Primary Key (PK)** in Incident_Offense
 - incident_id as Foreign Key (FK)** in Incident_Offense

9: Add Relationship Arrows (Visual Step)

- Go to **Insert → Shapes → Arrow**
- Draw arrows:
- From **incident_id (PK)** → **incident_id (CPK)**



Explanation: This shows how tables are connected.

Section 3 – Second Normal Form (2NF)

Background (What Second Normal Form Does):

Second Normal Form (2NF) “removes partial dependencies” from a table. This means that any attribute that depends on only part of a “composite key” should be moved into a separate table (Decomplexify).

In this stage, related information such as location data is separated, so the database becomes more efficient and easier to maintain.

10: Identify Partial Dependencies

- a. Look for attributes that depend on only part of a key
 - i. Example: Location data (*City, State, Zip Code*) depends on **location**, not incident

11: Create the “Location” Table

Create a new table:

[Location]

- location_id (PK)
- City
- State

- Zip Code
- Police District Number
- Police District Name

12: Update the “Incidents” Table

Modify Incidents:

- a. Remove: City, State, Zip Code
- b. Add: [location_id](#) (FK)

13: Create the “Offense” Table

Move offense data into a new table:

Offense

- [offense_id](#) (PK)
- Offense Code
- Crime_against
- Crime_name
- Crime_category

14: Update “Incident Offense” Table

Now it becomes a linking table:

Incident_Offense

- a. [incident_id](#) (FK CPK)
- b. [offense_id](#) (FK CPK)
 - i. **Note:** This is a **composite primary key (CPK)** (*Means both fields together uniquely identify a record*)

15: Add Relationship Arrows

(Refer back to Step 9 for instructions to find arrow shapes).

- a. Draw arrows:
 - i. [Incident_id](#) PK → Incident_Offense Table
 - ii. [Offense_id](#) PK → Incident_Offense Table
 - iii. [Location_id](#) PK → Incidents Table

	A	B	C	D	E	F	G	H	I
1	Incident_ID	Offence Code	Victims	Crime Against	Crime_Name	Crime_Category	Police District Name	Police District Number	City
2	201087100	1315	1	Crime Against Per	Aggravated Assa	Aggravated	GERMANTOWN	5D	GERMANTO
3									
4					NF1			NF2	
5	Data:				Incidents			Incidents	
6	Incident ID				incident_id (PK)			incident_id (PK)	
7	Offence Code				Victims			location_id (FK)	
8	Victims				Police District Number			Victims	
9	Crime_Against				Police District Name			Agency	
10	Crime_Name				City				
11	Crime_Category				State			Location	
12	Police District Name				Zip Code			location_id (PK)	
13	Police District Number				Agency			City	
14	City							State	
15	State				Incident Offense			Zip Code	
16	Zip Code				offense_id (PK)			Police District Number	
17	Agency				incident_id (FK CPK)			Police District Name	
18					Offence Code			Incident Offense	
19					Crime_against			incident_id (FK CPK)	
20					Crime_name			offense_id (FK CPK)	
21					Crime_category				
22								Offense	
23								offense_id (PK)	
24								Offence Code	
25								Crime_against	
26								Crime_name	
27								Crime_category	
28									

Explanation: Tables are connected; 2NF eliminates the partial dependencies, where an attribute depends on only part of a composite key.

Section 4 – Third Normal Form (3NF)

Background (What Second Normal Form Does):

Third Normal Form (3NF) removes “transitive dependencies”. This means that non-key attributes should depend only on the “primary key” and not on other non-key fields (Decomplexify).

In this step, the database becomes more organized by separating information such as agency and district data into their own tables.

16: Identify Transitive Dependencies

- a. Look for attributes depending on non-key fields
 - i. Example: Agency data (*agency*) depends on **agency**. Not incidents; District data (*Police District Number, Police District Name*) depend on **district**. Not incidents.

17: Create “Agency” Table

Move agency data to the Agency table.

[Agency]

- agency_id (PK)
- agency

18: Create “District” Table

- Move district data to the District table from the Location Table.
 - The attributes **Police District Number** and **Police District Name** are removed because they do not directly depend on location_id. This change removes transitive dependencies, which is the goal of Third Normal Form.

[District]

- district_id (PK)
- Police District Number
- Police District Name

19: Updating “Location” Table

Modify the existing **Location** table to remove district-related attributes and replace them with a foreign key.

Location Table Now:

- location_id (PK)
- district_id (FK)
- City
- State
- Zip Code

20: Create Relationship Between District and Location

- Insert an arrow using Insert → Shapes → Arrow
- Draw the arrow: **district_id (PK)** in **District** → **district_id (FK)** in **Location**

This relationship shows that:

- Each **Location** belongs to one district.
- Each **District** can have many locations.

21: Update “Incidents” Table

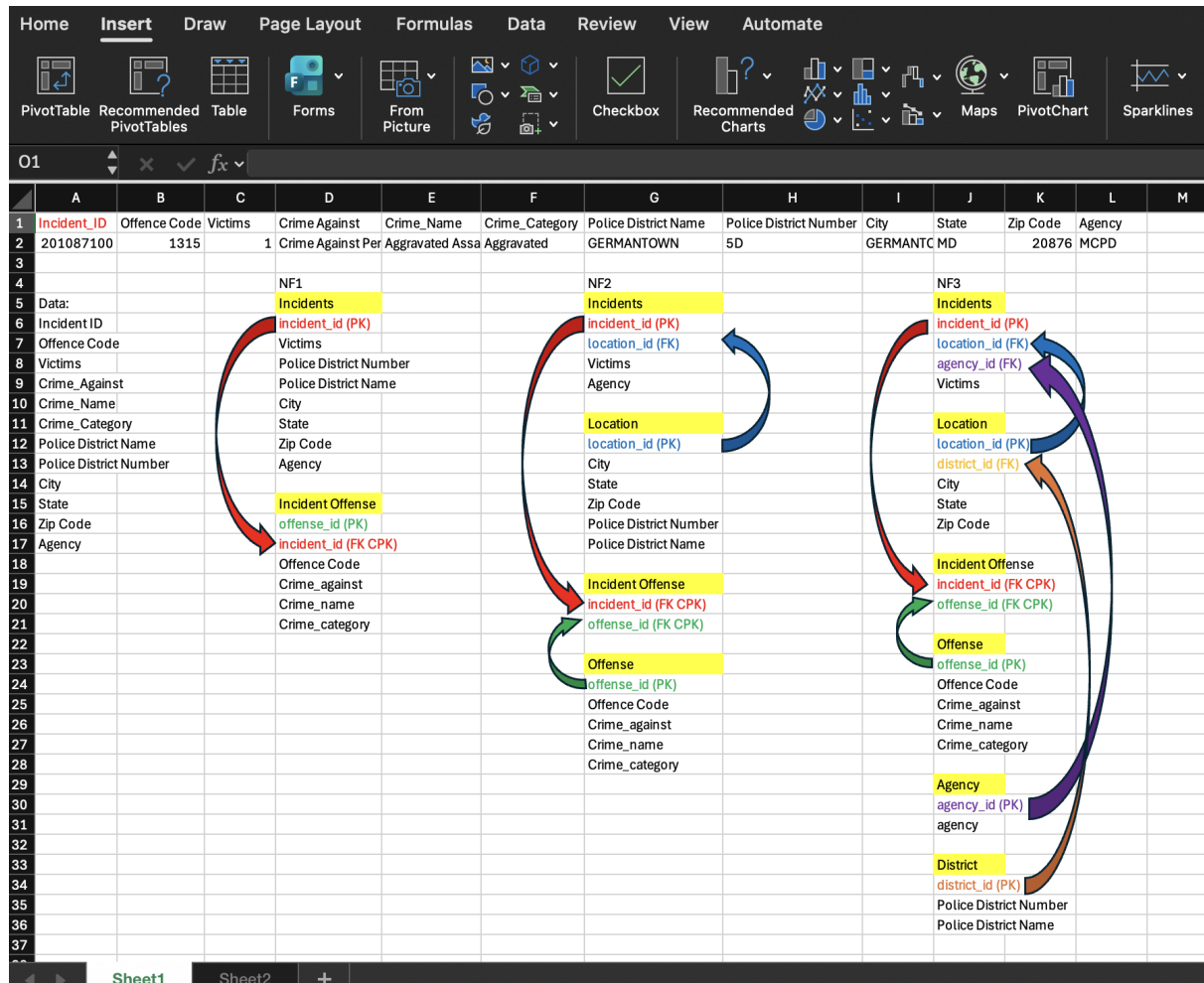
[Incidents]

- incident_id (PK)
- location_id (FK)
- agency_id (FK)
- Victims

22: Add Final Relationships

Draw arrows for:

- agency_id → Incidents Table
- district_id → Location Table
- location_id → Incidents Table
- offense_id → Incident_Offense Table
- incident_id → Incident_Offense Table



Final Result

At this point, the dataset should be fully normalized to the Third Normal Form.

After completing all these steps, your dataset should have been transformed from a single unstructured table into multiple different related tables that meet the requirements of Third Normal Form. This is the structured format that can now be used to build relational databases in software such as MySQL and get you started on implementing data from each individual table.

Congratulations! You now have...

- A fully normalized dataset up to 3NF
- Multiple related tables
- Proper PK/FK relationships
- Reduced redundancy and improved structure

Additional Resources:

The following resources were consulted during the development of this guide and may be useful for readers who wish to learn more about database normalization, relational database design, and data management.

Database Normalization. (n.d.). Wwww.w3schools.in.

<https://www.w3schools.in/dbms/database-normalization>

Decomplexify. (2021, November 21). *Learn Database Normalization - 1NF, 2NF, 3NF, 4NF, 5NF.* Wwww.youtube.com. https://www.youtube.com/watch?v=GfQaEYEc8_8

Dues, C. (2025, July 7). *Data Literacy: Essential Skill for Students - Why Data Literacy is an Essential Skill for Students in Today's World.* Population Education.

<https://populationeducation.org/why-data-literacy-is-an-essential-skill/>

GeeksforGeeks. (2015, July 2). *Introduction of Database Normalization.* GeeksforGeeks.

<https://www.geeksforgeeks.org/dbms/introduction-of-database-normalization/>

Jonker, A., Russi, E., & Scapicchio, M. (2026, January 21). *Data Management Hub.* Ibm.com.

<https://www.ibm.com/think/topics/data-management-guide#605511093>

Montgomery County Government. (2026). *Crime: Open data from Montgomery County Police Department.*

https://data.montgomerycountymd.gov/Public-Safety/Crime/icn6-v9z3about_data

Shiksha Online. (2022, April 30). *Introduction to Normalization – SQL Tutorial - Shiksha Online.* Shiksha.com; Shiksha Online.

<https://www.shiksha.com/online-courses/articles/introduction-to-normalization-sql-tutorial>

Vinayak Baranwal. (2025, July 22). *Database Normalization: 1NF, 2NF, 3NF & BCNF*

Examples. Digitalocean.com; DigitalOcean.

<https://www.digitalocean.com/community/tutorials/database-normalization>